SPI SWITCH: MASTER + SLAVE

Pure bare-metal C on two NUCLEO-F446RE boards

Part 1: the story | Part 2: night-before-exam deep dive

What the program does, in one breath

Press the USER button (PC13) on the **MASTER** board. It sends the byte `'S'` over SPI. The **SLAVE** board blinks its LED 5 times, then loads the acknowledge byte `'D'`. The master reads `'D'` back with a second transfer and blinks its own LED twice to confirm. Idle: both LEDs dark.

Part	What you get	How to use it
1. The Story	The whole program as a short tale, with each scene tagged to the code it represents	Read once, slowly. Builds the mental movie.
2A. Big picture	Wiring, roles, SPI theory, memory map	Skim first on exam night.
2B. Bit-level decode	Every register line decoded to bits and hex	Memorize the masks.
2C. Code walkthrough	Master and slave, function by function, plus a full timeline	Be able to redraw the timeline from memory.
2D. Weak points + debug	Design flaws, improvements, troubleshooting table	Great for 'suggest an improvement' questions.
2E. Q and A + cram card	17 likely exam questions with model answers, one-page cheat sheet	Last 30 minutes before sleep.

How to read the callouts

EXAM TIP = something examiners love to ask. **TRAP** = a subtle behaviour that causes real bugs.

PART 1 - THE STORY

The Captain, the Sleepy Guard, and the Sealed Envelope

Two small castles stand on opposite hills. One is ruled by **Captain Master**, who owns a big button. The other is watched by the **Sleepy Guard** (the Slave), who owns a single lantern. Between the castles run three pipes and one shared floor. That is the whole kingdom. Here is what happens when the Captain presses his button.

Character	Real thing	Role in the story
Captain Master	The MASTER board	Owns the button (PC13) and the drum
Sleepy Guard	The SLAVE board	Owns the lantern (LED, PA5); only acts when drummed
The Drum	SCK = PB13	Clock made by the Captain. One beat moves exactly one bit
Outgoing pipe	MOSI = PB15	Master Out, Slave In. Captain to Guard
Return pipe	MISO = PB14	Master In, Slave Out. Guard to Captain
The shared floor	GND	Both castles agree what 'zero volts' means
Sealed envelope	Slave's SPI2_DR (TX side)	The reply, which must be sealed BEFORE the drum rolls
Secret words	`'S'` = 0x53, `'D'` = 0x44	Request word and Done word

Chapter 1 - Switching on the castles

A building with no electricity ignores every order, so the first job is power. The Captain switches on three buildings (GPIOA for the lantern, GPIOB for the pipes, GPIOC for the button) and the messenger office, SPI2. The Guard needs only two buildings, because he has no button. Next, each pipe is given a new job. PB13, PB14 and PB15 stop being ordinary pins and become SPI2 pipes. The Captain also tells the return pipe to lean towards silence (a pull-down), so if the Guard ever says nothing he hears quiet, not random crackle. His button gets a pull-up, so it reads 'not pressed' until a finger drags it to ground. Finally, ranks are assigned: the Captain wears the master badge and sets a slow, steady drum speed; the Guard wears the slave badge.

[code: RCC enables, GPIOx_MODER, GPIOB_AFRH, PUPDR, SPI2_CR1]

Chapter 2 - The blank envelope

Here is the law of this kingdom: **the Guard can never speak first**. Only the Captain's drum makes anything move, and every roll of eight beats swaps one byte each way at the same moment. So the Guard does the only thing he can. He slides a blank envelope (0x00) into his outbox, so that whatever the Captain's first drum-roll fetches back, it is blank. Then he dozes at the door, waiting for his inbox to light up.

[code: slave writes SPI2_DR = 0x00, then polls RXNE]

Chapter 3 - The button press

The Captain glances at his button every five milliseconds. The moment it goes from 'up' to 'down' (a fresh press, not a held finger) he waits twenty milliseconds in case it was only a spark, then checks again. Real press. He beats the drum eight times while whispering the secret word `'S'`. On those same eight beats the Guard's blank envelope slides back along the return pipe. The Captain does not care about it and throws it away.

[code: edge detect, delay_ms(20), spi_xfer('S')]

Chapter 4 - The Guard wakes

The eight beats end and the Guard's inbox lights up. He reads `'S'` and knows his duty: he lights his lantern five times, two full seconds of blinking. Only when he is finished does he seal a *new* envelope containing `'D'` (for Done) and slide it into the outbox, ready for the next drum-roll. The Captain cannot see any of this. There is no phone line, and the Guard cannot call out.

[code: if (b == 'S') { blink(5); SPI2_DR = 'D'; }]

Chapter 5 - Blind patience

With no way to ask 'are you done yet?', the Captain gambles on time. He waits 2.5 seconds, which is the Guard's 2 seconds of blinking plus half a second of safety margin. Then he beats the drum again, this time carrying a meaningless blank message. The message is not the point. The drum is. Without a drum-roll, the Guard's envelope has no way to travel. The envelope arrives: `'D'`. The Captain flashes his own lantern twice. Mission confirmed.

[code: delay_ms(2500); ack = spi_xfer(0x00); if (ack == 'D') blink_little();]

Chapter 6 - Back to sleep

The Guard receives the blank message 0x00. It is not an `'S'`, so he does the safe thing and replaces his envelope with a fresh blank one. Both castles go dark. The kingdom is ready for the next press.

[code: else { SPI2_DR = 0x00; }]

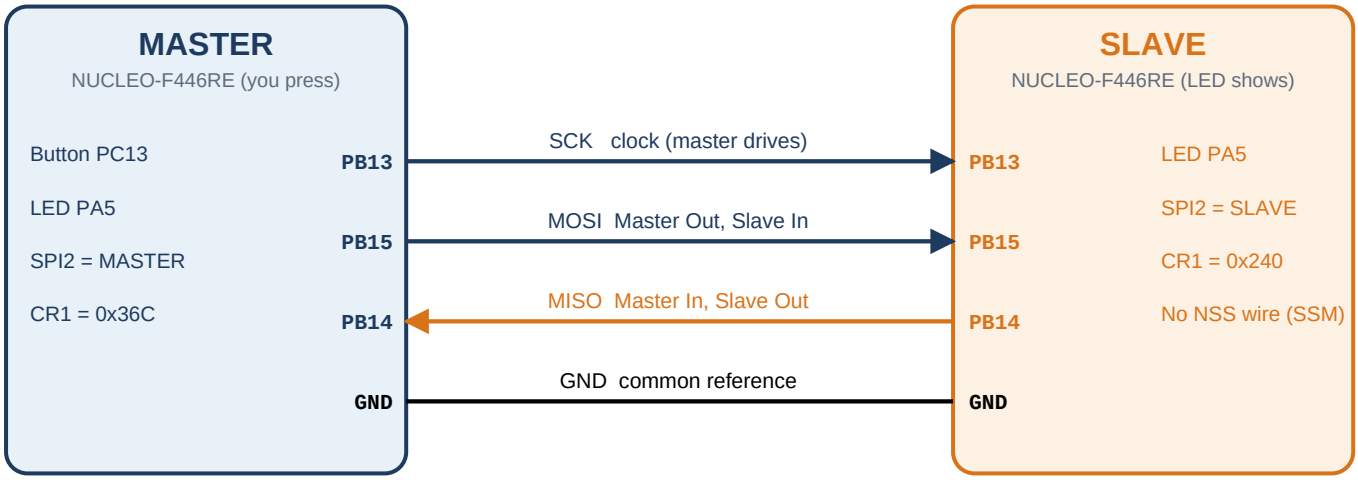
> **THE MORAL - three laws of SPI hidden in this story**
>
> 1) The **master owns the clock**: the slave can never start a conversation.
> 2) **Every transfer is a swap**: to read a byte you must send one (a dummy byte if you have nothing to say).
> 3) The slave's answer is **whatever was preloaded before the transfer began**, so replies always arrive one transfer late. That is exactly why the acknowledge needs a second transfer.

PART 2 - NIGHT-BEFORE-EXAM DEEP DIVE

2A. Big picture

60-SECOND SUMMARY

Two NUCLEO-F446RE boards, SPI2, 3 signal wires + GND, no chip-select wire. Master (CR1 = 0x36C): 250 kHz, mode 0, 8-bit, MSB first. Slave (CR1 = 0x240): software NSS, always selected. Press -> master sends 'S' (0x53) -> slave blinks x5 (2000 ms) -> slave loads 'D' (0x44) -> master waits 2500 ms, sends dummy 0x00, receives 'D' -> master blinks x2. Slave always keeps 0x00 preloaded as its default reply.



Wire SCK to SCK, MOSI to MOSI, MISO to MISO (straight through, NOT crossed) and join the grounds. Unlike UART there is no TX-to-RX crossing: the pin names already describe direction from the master's point of view.

Roles side by side

Item	MASTER project	SLAVE project
Clocks enabled	GPIOA, GPIOB, GPIOC, SPI2	GPIOA, GPIOB, SPI2 (no button, no GPIOC)
PA5	Output, LED (2 confirm blinks)	Output, LED (5 request blinks)
PB13 / PB14 / PB15	AF5: SCK out, MISO in, MOSI out	AF5: SCK in, MISO out, MOSI in
Pull resistors	PB14 pull-down, PC13 pull-up	None configured
SPI2_CR1	0x36C = MSTR + BR/64 + SSM + SSI + SPE	0x240 = SSM + SPE (MSTR = 0)
Main loop	Polls button every 5 ms; runs one transaction per press	Blocks on RXNE; reacts to each received byte
Initiates transfers?	YES, always	NEVER (cannot)

SPI theory you must be able to write from memory

- **4 signals:** SCK (clock), MOSI (master to slave data), MISO (slave to master data), NSS/CS (chip select, active low). Here NSS is replaced by software NSS on both boards, so only 3 wires + GND are used.
- **Full duplex shift-register ring:** master's 8-bit shift register and slave's 8-bit shift register are connected in a ring. After 8 clock pulses the two bytes have been exchanged. Sending and receiving happen *simultaneously*, never separately.
- **Mode:** CPOL = 0 (SCK idles low), CPHA = 0 (data sampled on the first, rising, edge) = **SPI mode 0**. Master and slave MUST use the same mode. Both boards leave CPOL/CPHA at 0 so they match.
- **Frame:** 8 bits (DFF = 0), MSB first (LSBFIRST = 0).
- **Baud rate:** $f_{SCK} = f_{PCLK} / 2^{(BR+1)}$. SPI2 sits on APB1. BR = 5 -> divide by 64 -> 16 MHz / 64 = **250 kHz** (assuming reset-default HSI 16 MHz, because SystemInit is an empty stub). One bit = 4 us, one byte = 32 us.

- **Flow control:** SPI has none built in. The slave must have its reply ready *before* the master clocks it out, which is why this demo preloads DR and uses a timed wait.

> **EXAM TIP - SPI vs UART vs I2C**
> SPI: synchronous, full duplex, separate data lines, master-driven clock, fastest, needs one CS per slave. UART: asynchronous (no clock wire), agreed baud rate, start/stop bits, point to point. I2C: synchronous, 2 wires (SDA + SCL) shared by many devices, addressed, half duplex, slower, needs pull-ups.

2B. Bit-level decode of every register line

Memory map (base + offset = address)

Macro in code	Address	Base + offset	Bus
RCC_AHB1ENR	0x40023830	RCC 0x40023800 + 0x30	AHB1 enable reg (GPIO clocks)
RCC_APB1ENR	0x40023840	RCC 0x40023800 + 0x40	APB1 enable reg (SPI2 clock)
GPIOA_MODER	0x40020000	GPIOA 0x40020000 + 0x00	AHB1
GPIOA_ODR	0x40020014	GPIOA + 0x14	AHB1
GPIOB_MODER	0x40020400	GPIOB 0x40020400 + 0x00	AHB1
GPIOB_PUPDR	0x4002040C	GPIOB + 0x0C	AHB1
GPIOB_AFRH	0x40020424	GPIOB + 0x24	AHB1 (AF for pins 8-15)
GPIOC_PUPDR	0x4002080C	GPIOC 0x40020800 + 0x0C	AHB1
GPIOC_IDR	0x40020810	GPIOC + 0x10	AHB1 (read pin levels)
SPI2_CR1	0x40003800	SPI2 0x40003800 + 0x00	APB1
SPI2_SR	0x40003808	SPI2 + 0x08	APB1
SPI2_DR	0x4000380C	SPI2 + 0x0C	APB1

Why the macros look like this

```
#define SPI2_DR (*(volatile unsigned int *)0x4000380C)
          |        |        |            |
          |        |        |            +-- fixed hardware address
          |        |        +--------------- treat the number as a pointer to 32-bit data
          |        +------------------------ volatile: compiler must really read/write every time
          +------------------------------- leading * : dereference, so SPI2_DR acts like a variable
```

EXAM TIP - why volatile?

Hardware registers change without the program touching them (RXNE flips by itself). Without volatile the compiler may cache the value and turn `while(!(SPI2_SR&1));` into an infinite loop, or delete the empty `for(...);` delay loop entirely.

Clock enables

Line	Bits set	Meaning
`RCC_AHB1ENR |= (1<<0)|(1<<1)|(1<<2)`	bit0, bit1, bit2	GPIOAEN, GPIOBEN, GPIOCEN. Master uses all three; slave sets only bits 0 and 1.
`RCC_APB1ENR |= (1<<14)`	bit14	SPI2EN. A peripheral whose clock is off ignores all register writes.

GPIO configuration with mask math

Rule: `MODER` uses 2 bits per pin at [2n+1 : 2n]. `AFRH` uses 4 bits per pin for pins 8-15 at [4(n-8)+3 : 4(n-8)]. `PUPDR` uses 2 bits per pin: 00 none, 01 pull-up, 10 pull-down. MODER values: 00 input, 01 output, 10 alternate function, 11 analog.

Register	Clear mask	Set value	Result
GPIOA_MODER (PA5)	~(3<<10) = clears bits 11:10	1<<10 = 0x00000400	PA5 = output (01), the LED
GPIOB_MODER (PB13/14/15)	3<<26 | 3<<28 | 3U<<30 = 0xFC000000	2<<26 | 2<<28 | 2U<<30 = 0xA8000000	bits 31:26 = 10 10 10, all three pins alternate function
GPIOB_AFRH	0xF<<20 | 0xF<<24 | 0xFU<<28 = 0xFFF00000	5<<20 | 5<<24 | 5U<<28 = 0x55500000	PB13, PB14, PB15 each = 0101 = AF5 = SPI2
GPIOB_PUPDR (PB14)	3<<28 = clears bits 29:28	2<<28 = 0x20000000	MISO has a pull-down (master only)
GPIOC_PUPDR (PC13)	3<<26 = clears bits 27:26	1<<26 = 0x04000000	button pin has a pull-up (master only)

Every line uses the same pattern: `REG &= ~MASK` then `REG |= VALUE`. That is the **clear-then-set** read-modify-write, which changes only the target field and leaves neighbouring pins untouched.

> **TRAP - why 3U, 2U, 0xFU, 5U on the top field?**
> Shifting a signed `int` into bit 31 is undefined behaviour in C. The `U` suffix makes the constant unsigned so the shift is well defined. Only the pin-15 terms need it, because they reach bit 31.

SPI2_CR1 decode (the most asked register)

Bit	Name	Master	Slave	Meaning
0	CPHA	0	0	Capture on first clock edge (with CPOL = 0: sample on rising edge)
1	CPOL	0	0	SCK idles LOW
2	MSTR	**1**	0	1 = master mode, 0 = slave mode
5:3	BR[2:0]	**101**	000 (unused)	Baud divider: 000=/2, 001=/4, 010=/8, 011=/16, 100=/32, 101=/64, 110=/128, 111=/256
6	SPE	**1**	**1**	SPI enable. Set LAST, after configuration
7	LSBFIRST	0	0	0 = MSB first
8	SSI	**1**	0	Internal NSS level when SSM = 1. Master needs it HIGH (1)
9	SSM	**1**	**1**	Software slave management: ignore the NSS pin, use SSI instead
10	RXONLY	0	0	0 = full duplex
11	DFF	0	0	0 = 8-bit frame
15	BIDIMODE	0	0	0 = 2-line unidirectional (MOSI and MISO separate)

```
MASTER: (1<<2)|(5<<3)|(1<<9)|(1<<8)|(1<<6) = 0x004+0x028+0x200+0x100+0x040 = 0x36C
SLAVE : (1<<9)|(1<<6)                      = 0x200+0x040                   = 0x240
```

EXAM TIP - SSM and SSI explained in two lines

With SSM = 1 the chip ignores the physical NSS pin and uses the SSI bit as the NSS level. **Slave:** SSI = 0 means 'NSS is low = I am selected, always', so no CS wire is needed. **Master:** SSI must be 1; if the internal NSS were low, the master would think another master is selecting it, raise a **mode fault (MODF)** and drop back to slave mode.

SPI2_SR flags

Bit	Flag	Meaning	Used how in this code
0	**RXNE**	Receive buffer Not Empty: a byte has arrived. Reading DR clears it	**Read** DR only when RXNE = 1
1	**TXE**	Transmit buffer Empty: safe to write a new byte to DR	**Write** DR only when TXE = 1
5	MODF	Mode fault (master saw NSS low)	Not used (this is what SSI = 1 prevents)
6	OVR	Overrun: new byte arrived before the old one was read	Not checked. Risk if slave is busy blinking
7	BSY	SPI busy shifting	Not used. RXNE polling already guarantees the frame ended

Mnemonic: **Write when TXE, Read when RXNE.** (TXE = Transmit buffer Empty, so I may fill it. RXNE = Receive Not Empty, so there is something to read.)

Pin cheat table

Pin	Job	MODER bits	AFRH bits	Other
PA5	LED output	11:10 = 01	-	ODR bit 5 (set = on)
PB13	SPI2_SCK	27:26 = 10	23:20 = 0101	AF5
PB14	SPI2_MISO	29:28 = 10	27:24 = 0101	PUPDR 29:28 = 10 (master)
PB15	SPI2_MOSI	31:30 = 10	31:28 = 0101	AF5
PC13	USER button (active low)	input (reset default)	-	PUPDR 27:26 = 01, IDR bit 13

2C. Code walkthrough

Boilerplate both files share

```
void SystemInit(void) __attribute__((weak));        void SystemInit(void) { }
void __libc_init_array(void) __attribute__((weak)); void __libc_init_array(void) { }
```

The startup file calls `SystemInit` and `__libc_init_array` before `main`. Defining empty **weak** versions satisfies the linker without CMSIS or newlib init, and a real implementation elsewhere could still override them. Because SystemInit is empty, the clock tree stays at reset defaults (HSI 16 MHz), which is what the delay loops and the 250 kHz figure assume.

```
static void delay_ms(unsigned int ms) {
    for (unsigned int m = 0; m < ms; m++)
        for (volatile unsigned int i = 0; i < 1300; i++);   // ~1 ms at 16 MHz (tuned by trial)
}
```

Software busy-wait. `volatile` stops the compiler from deleting the empty loop. It is approximate and changes if the clock or optimization level changes. It also blocks the CPU completely.

MASTER: helper functions

Function	Code idea	What to say in the exam
`pressed()`	`(GPIOC_IDR & (1<<13)) == 0`	The USER button is wired active-low: pin reads 0 when pressed. Returns 1 if pressed.
`blink_little()`	2 x (ON 120 ms, OFF 120 ms) on PA5	Success indicator = 480 ms of visible feedback.
`spi_xfer(b)`	wait TXE, DR = b, wait RXNE, return DR	One full-duplex byte exchange. Returns what the slave shifted out DURING this transfer.

```
static unsigned char spi_xfer(unsigned char b) {
    while (!(SPI2_SR & (1 << 1)));  SPI2_DR = b;        // 1) TXE=1 ? then load TX byte
    while (!(SPI2_SR & (1 << 0)));  return (unsigned char)SPI2_DR; // 2) wait RXNE, read reply
}
```

Master main loop, annotated

```
int prev = 0;
while (1) {
    int now = pressed();                // 1 while finger is down
    if (now && !prev) {                 // rising edge of 'pressed' = a NEW press only
        delay_ms(20);                   // debounce
        if (pressed()) {                // still down -> genuine press
            spi_xfer('S');              // TX 0x53 | RX = slave's preloaded 0x00 (thrown away)
            delay_ms(2500);             // blind wait: slave needs 2000 ms to blink
            unsigned char ack = spi_xfer(0x00);   // TX dummy 0x00 | RX = 'D' (0x44)
            if (ack == 'D') blink_little();       // confirmed
        }
    }
    prev = now;                         // remember for edge detection
    delay_ms(5);                        // poll every 5 ms
}
```

EXAM TIP - why edge detection (`now && !prev`)?

Without it, holding the button would restart the sequence over and over. With it, one physical press = exactly one transaction, even if the finger stays down.

SLAVE: main loop, annotated

```
RCC_AHB1ENR |= (1 << 0) | (1 << 1);        // GPIOA + GPIOB (no button, so no GPIOC)
RCC_APB1ENR |= (1 << 14);                  // SPI2 clock
/* PA5 output; PB13/14/15 AF5 -- same masks as the master */
SPI2_CR1 = (1 << 9) | (1 << 6);            // SSM=1 (SSI=0 -> always selected), SPE=1, MSTR=0 -> slave

while (!(SPI2_SR & (1 << 1)));  SPI2_DR = 0x00;   // PRELOAD the default reply

while (1) {
    while (!(SPI2_SR & (1 << 0)));         // wait RXNE: master has sent a byte
    unsigned char b = (unsigned char)SPI2_DR;      // read it (also clears RXNE)

    if (b == 'S') {
        blink(5);                          // 5 x (200 ON + 200 OFF) = 2000 ms
        while (!(SPI2_SR & (1 << 1)));
        SPI2_DR = 'D';                     // acknowledge, ready for master's NEXT read
    } else {
        while (!(SPI2_SR & (1 << 1)));
        SPI2_DR = 0x00;                    // default reply (also re-arms after the dummy transfer)
    }
}
```

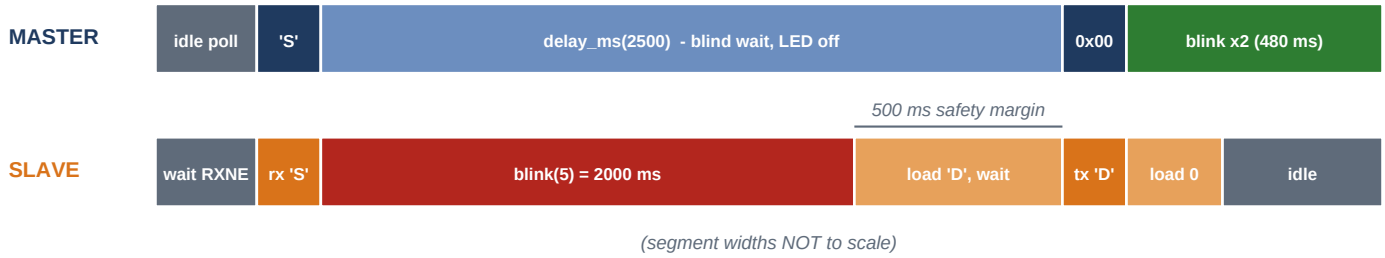
TRAP - the one-transfer-late reply

When the master sends 'S', the slave's shift register is already carrying the OLD preloaded byte (0x00). The new byte 'D' is only written AFTER the blink, so it can only leave during the **next** transfer. That is why the master needs a second transfer, and why the first return value is thrown away.

Why the else-branch matters

After the master's dummy transfer the slave receives 0x00, takes the else-branch and reloads 0x00. Without it, DR would be empty or stale, and the next transaction's first return byte would be undefined.

Full timeline of one button press



(segment widths NOT to scale)

#	When	Master	Wires (MOSI / MISO)	Slave	Slave TX byte
0	Power-up	Init, polls every 5 ms	SCK idle low	Init, loads DR = 0x00, waits RXNE	0x00 ready
1	Press	Edge seen, 20 ms debounce, re-check	idle	Still waiting	0x00
2	t0	spi_xfer('S'): 8 clocks	0x53 / 0x00	RXNE=1, reads 'S'	0x00 just left
3	t0 .. t0+2000 ms	delay_ms(2500)	idle	blink(5)	none loaded
4	t0+2000 ms	still waiting	idle	TXE ok, DR = 'D'	'D' ready
5	t0+2500 ms	spi_xfer(0x00): 8 clocks	0x00 / 0x44	RXNE=1, reads 0x00	'D' left
6	after	ack == 'D', so blink_little() (480 ms)	idle	else-branch, DR = 0x00	0x00
7	idle	back to polling	idle	back to waiting	0x00

Timing numbers to memorize

Quantity	Value	How
SCK frequency	250 kHz	16 MHz / 64 (BR = 101)
One bit / one byte	4 us / 32 us	1 / 250 kHz; times 8
Slave blink time	2000 ms	5 x (200 ON + 200 OFF), BLINK_MS = 200
Master wait	2500 ms	Fixed delay_ms(2500)
Safety margin	500 ms	2500 - 2000
Master success blink	480 ms	2 x (120 + 120)
Debounce / poll	20 ms / 5 ms	delay_ms values

2D. Weak points, improvements, debugging

Design weaknesses (great for 'how would you improve this?')

Weakness	Why it matters	Better design
Blind 2500 ms delay (open loop)	If the slave blinks longer than 2.5 s, the master reads 0x00 and sees no ack. Timing is guessed, not confirmed.	Master polls: send 0x00 repeatedly until reply is 'D' (with a timeout). Or add a READY GPIO line from the slave.
Slave blocks inside blink()	A byte arriving during the 2 s blink can cause an overrun (OVR) and be lost.	Use the SPI RXNE interrupt, or do the blink with a timer/state machine so SPI is always serviced.
No timeouts on while-loops	A wrong or missing wire hangs both boards forever in a polling loop.	Add a timeout counter or a watchdog.
No chip select (software NSS)	If one clock glitch shifts the bit alignment, the pair stays out of sync forever. NSS would reset the frame.	Use hardware NSS (PB12) or a GPIO chip-select driven low for each transfer.
Fixed clock assumption	Delay loops and baud rate are only right at HSI 16 MHz.	Use a hardware timer (SysTick) for delays; configure the clock tree explicitly.
Polling everywhere	CPU is 100% busy waiting.	Interrupts or DMA, sleep between events.
Minimal protocol	One command, one ack, no error codes, no checksum.	Add a command byte, status/error byte and optional CRC.

Troubleshooting table

Symptom	Likely cause	Check
Slave never blinks	SCK/MOSI wrong or missing; no common GND; wrong project flashed; SPI2 clock not enabled	Wiring straight (not crossed), GND joined, flash MASTER on the button board and SLAVE on the other
Slave blinks, master never confirms	MISO wire missing (pull-down makes master read 0x00); ack loaded too late; slave blink longer than 2.5 s	MISO PB14 to PB14; BLINK_MS x 10 must stay below 2500
Master LED never blinks and board seems frozen	Hung in a while(!RXNE) or while(!TXE) loop: SPE not set, pins not AF, SPI2 clock off	Verify CR1 = 0x36C and AFRH = 0x555 in the 20-31 bit range
Button does nothing	GPIOC clock off, or pressed() logic inverted	RCC_AHB1ENR bit 2; button is active LOW
Random behaviour	Floating wires, no ground, long jumpers	Short wires, shared GND
Works once, then odd	Slave powered or reset after master started clocking	Reset the slave first, then the master

2E. Likely exam questions with model answers

Q1. Why does the master send a dummy byte (0x00) to read the acknowledge?

A. SPI is full duplex and the master generates the clock. The slave can only shift data out while clock pulses arrive, so the master must clock a byte to receive one. The byte it sends has no meaning, so it is a dummy.

Q2. Why does the first `spi_xfer('S')` transfer not return the ack?

A. The slave's reply for a transfer must be preloaded before it starts. During that transfer the slave shifts out the old preloaded 0x00. It only loads 'D' after blinking, so 'D' leaves on the next transfer.

Q3. What are SSM and SSI, and why is SSI = 1 on the master but 0 on the slave?

A. SSM = 1 makes the hardware use the SSI bit instead of the NSS pin. Slave with SSI = 0: internal NSS low, so it is always selected. Master with SSI = 1: internal NSS high, so no mode fault (MODF) occurs.

Q4. Which SPI mode is used and how do you know?

A. Mode 0: CPOL = 0 (idle low) and CPHA = 0 (sample on first/rising edge). Both bits are 0 in CR1.

Q5. Calculate the SPI clock frequency.

A. f = f_PCLK / 2^(BR+1) = 16 MHz / 2^6 = 16 MHz / 64 = 250 kHz. One byte takes 8 / 250 kHz = 32 us.

Q6. Why must the peripheral clocks be enabled before configuring GPIO and SPI?

A. A peripheral without its clock does not respond to register writes, so all later configuration would be silently lost.

Q7. Why AF5 for PB13, PB14, PB15?

A. The alternate-function table in the datasheet maps SPI2_SCK/MISO/MOSI on these pins to AF5. AFRH holds the 4-bit AF number for pins 8-15.

Q8. Why a pull-down on MISO and a pull-up on PC13?

A. MISO pull-down gives a defined 0 if the slave is missing or not driving, instead of floating noise. The button connects PC13 to GND when pressed, so a pull-up makes the idle level 1 and the pressed level 0.

Q9. What does `REG &= ~MASK; REG |= VALUE;` do and why use it?

A. Clear-then-set read-modify-write. It changes only the chosen bit field and leaves other pins' settings intact.

Q10. Why `3U<<30` and `0xFU<<28` instead of plain 3 and 0xF?

A. Shifting a signed int into the sign bit (bit 31) is undefined behaviour. Unsigned constants avoid it.

Q11. What are TXE and RXNE? When do you use each?

A. TXE (SR bit 1): transmit buffer empty, safe to write DR. RXNE (SR bit 0): a received byte is waiting, safe to read DR. Write when TXE, read when RXNE. Reading DR clears RXNE.

Q12. What happens if the master sends a byte other than 'S'?

A. The slave takes the else-branch: no blink, reloads 0x00. Nothing visible happens and the master will read 0x00 as the reply.

Q13. Why define empty weak SystemInit and __libc_init_array?

A. The startup code calls them. Empty weak stubs satisfy the linker without CMSIS/newlib setup, and can be overridden by real versions.

Q14. What is the role of `volatile` on the registers?

A. It forces the compiler to perform every read/write, since hardware values change independently of the program.

Q15. Name two problems with this design and fix them.

A. (1) Blind 2500 ms delay: poll with dummy bytes until 'D' or use a READY pin. (2) Slave blocked in blink(): use RXNE interrupt or a non-blocking blink. Also: no timeouts, no chip select.

Q16. What would happen if the slave's blink took 3 seconds?

A. The master's dummy transfer happens at 2.5 s, before 'D' is loaded. It would read the default 0x00, so no confirmation blink. The slave would then reload 'D' too late; the stale 'D' would come back on the NEXT press's first transfer.

Q17. Is SPI synchronous or asynchronous? Half or full duplex?

A. Synchronous (shared clock), and full duplex (MOSI and MISO carry data simultaneously).

ONE-PAGE CRAM CARD

Read this last, then sleep

Topic	Remember
Flow	Press -> 'S' (0x53) -> slave blinks x5 (2000 ms) -> loads 'D' (0x44) -> master waits 2500 ms -> dummy 0x00 -> reads 'D' -> blinks x2
Wiring	SCK-SCK, MOSI-MOSI, MISO-MISO, GND-GND. PB13 / PB15 / PB14. No NSS wire.
CR1	Master 0x36C = MSTR, BR=101, SSM, SSI, SPE. Slave 0x240 = SSM, SPE.
Baud	fPCLK / 2^(BR+1) = 16 MHz / 64 = 250 kHz; byte = 32 us
Mode	CPOL 0, CPHA 0 = mode 0, 8-bit, MSB first
Flags	SR bit0 RXNE (read), bit1 TXE (write), bit5 MODF, bit6 OVR, bit7 BSY
Pins	AF5 = SPI2. MODER pin n at 2n; AFRH pin n at 4(n-8); PUPDR pin n at 2n
Masks	MODER 0xFC000000 -> 0xA8000000; AFRH 0xFFF00000 -> 0x55500000
Clocks	AHB1ENR bit0/1/2 = GPIOA/B/C; APB1ENR bit14 = SPI2
Button	PC13 active low, pull-up, IDR bit 13 == 0 means pressed; edge detect + 20 ms debounce
Key trap	Slave reply is preloaded and arrives ONE transfer late; read requires write (dummy)
Weak points	Blind delay, blocking blink, no timeout, no CS, polling
Fixes	Poll-until-ack, READY pin, interrupts, hardware NSS, SysTick timing

7-STEP RECIPE FOR ANY BARE-METAL PERIPHERAL

1) Enable clock (RCC). 2) Set pin mode (MODER). 3) Select alternate function (AFR). 4) Set pulls (PUPDR). 5) Configure the peripheral (CR1). 6) Enable it (SPE) LAST. 7) Use it by polling flags: write on TXE, read on RXNE.

Mnemonics: **MOSI** = Master Out, Slave In. **MISO** = Master In, Slave Out. **Every write is a read**: SPI always swaps. **Slave is polite**: it never speaks until spoken to, and its answer was written before the question.